FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	CRUZ, KRISTA
Section:	AN41
Professor:	MS. TAN
Date:	10/02/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

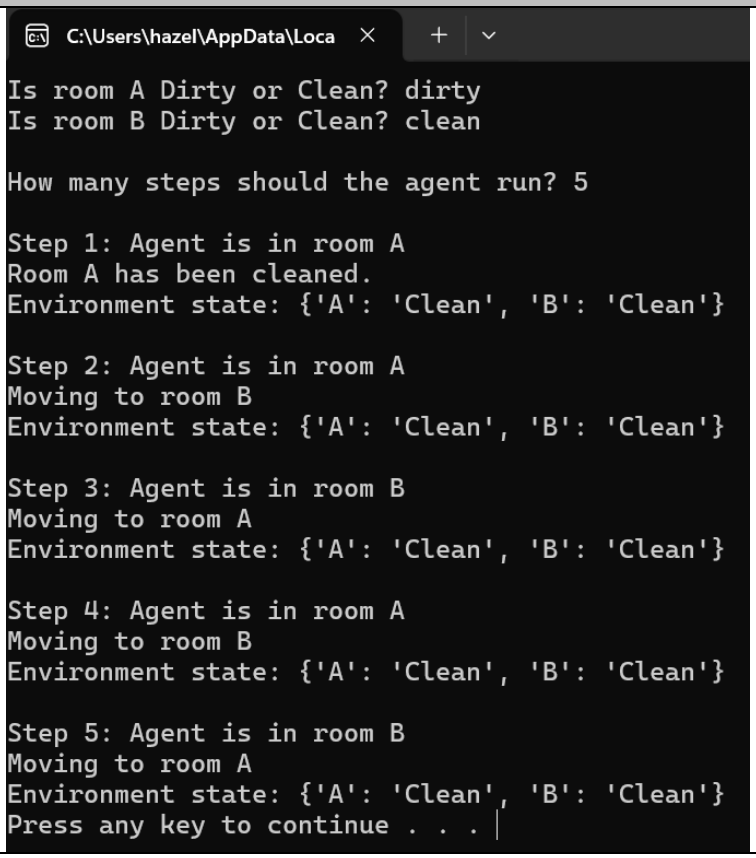
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

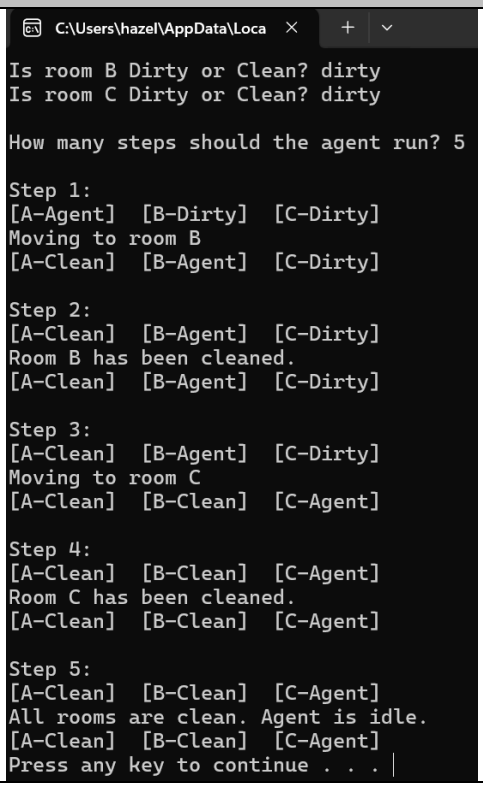
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
class VacuumAgent:
    def __init__(self, start_room="A"):
        self.location = start_room

    def perceive_and_act(self, environment):
        """
        Rule 1: If the current room is dirty, clean it.
        Rule 2: If the current room is clean, move to the other room.
        """
        action_taken = ""
        if environment[self.location] == "Dirty":
            environment[self.location] = "Clean"
            action_taken = "Cleaned"
        else:
            self.move(environment)
            action_taken = "Moved"
        return action_taken

    def move(self, environment):
        """Switch rooms between A and B."""
        rooms = list(environment.keys())
        if len(rooms) == 2:
            self.location = rooms[1] if self.location == rooms[0] else rooms[0]
        else:
            current_idx = rooms.index(self.location)
            self.location = rooms[(current_idx + 1) % len(rooms)]


def run_simulation():
    # set up the environment
    environment = {"A": "Clean", "B": "Clean"}

    print("--- Environment Setup ---")
    for room in environment:
        state = input(
            f"Enter initial state for Room {room} (Dirty/Clean): "
        ).strip().capitalize()
        if state in ["Dirty", "Clean"]:
            environment[room] = state
        else:
            print("Invalid input defaulting to Clean.")
            environment[room] = "Clean"
```

```
    agent = VacuumAgent(start_room="A")

    try:
        steps = int(input("How many steps should the simulation run? "))
    except ValueError:
        steps = 5

    print("\n--- Simulation Started ---")
    for step in range(1, steps + 1):
        print(f"\nStep {step}:")
        print(f"  - Agent's Current Location: Room {agent.location}")

        action = agent.perceive_and_act(environment)
        print(f"  - Action Taken: {action}")

        env_str = ", ".join([f"{room} = {state}" for room, state in environment.items()])
        print(f"  - Current State of Environment: {env_str}")

    # check if all rooms are clean
    all_clean = all(state == "Clean" for state in environment.values())
    if all_clean:
        print("\nAgent is done making rounds (all rooms are clean).")
    else:
        print("\nAgent finished steps, but dirty rooms still remain.")


if __name__ == "__main__":
    run_simulation()
```

Output:

```
--- Environment Setup ---
Enter initial state for Room A (Dirty/Clean): dirty
Enter initial state for Room B (Dirty/Clean): clean
How many steps should the simulation run? 3

--- Simulation Started ---

Step 1:
  - Agent's Current Location: Room A
  - Action Taken: Cleaned
  - Current State of Environment: A = Clean, B = Clean

Step 2:
  - Agent's Current Location: Room A
  - Action Taken: Moved
  - Current State of Environment: A = Clean, B = Clean

Step 3:
  - Agent's Current Location: Room B
  - Action Taken: Moved
  - Current State of Environment: A = Clean, B = Clean

Agent is done making rounds (all rooms are clean).
```

```
--- Environment Setup ---
Enter initial state for Room A (Dirty/Clean): dirty
Enter initial state for Room B (Dirty/Clean): dirty
How many steps should the simulation run? 2

--- Simulation Started ---

Step 1:
  - Agent's Current Location: Room A
  - Action Taken: Cleaned
  - Current State of Environment: A = Clean, B = Dirty

Step 2:
  - Agent's Current Location: Room A
  - Action Taken: Moved
  - Current State of Environment: A = Clean, B = Dirty

Agent finished steps, but dirty rooms still remain.
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. The agent follows two primary rules. One is if the room the agent is currently in is dirty, the agent cleans it. The other is if the current room is clean, the agent moves to the other room.
2. The agent checks the state of the room it is in. If the state is “Dirty”, it triggers the cleaning action. If the state is “Clean”, it bypasses cleaning and switches rooms.
3. If both rooms are already clean at the start, the agent will move back and forth between the rooms without changing the environment state until it reaches the steps that the simulation is set to run.
4. To handle more than two rooms, I would add the extra room/s to the environment list and update the movement code so the agent can cycle through all of them one by one.